

Introducción a Bases de Datos NoSQL

Bases de Datos II



Introducción

- Surge para perfeccionar las BD con exceso de datos, que necesitan un gran indexado de documentos
- Con el crecimiento de la web en tiempo real existía una necesidad de proporcionar información procesada a partir de grandes volúmenes de datos

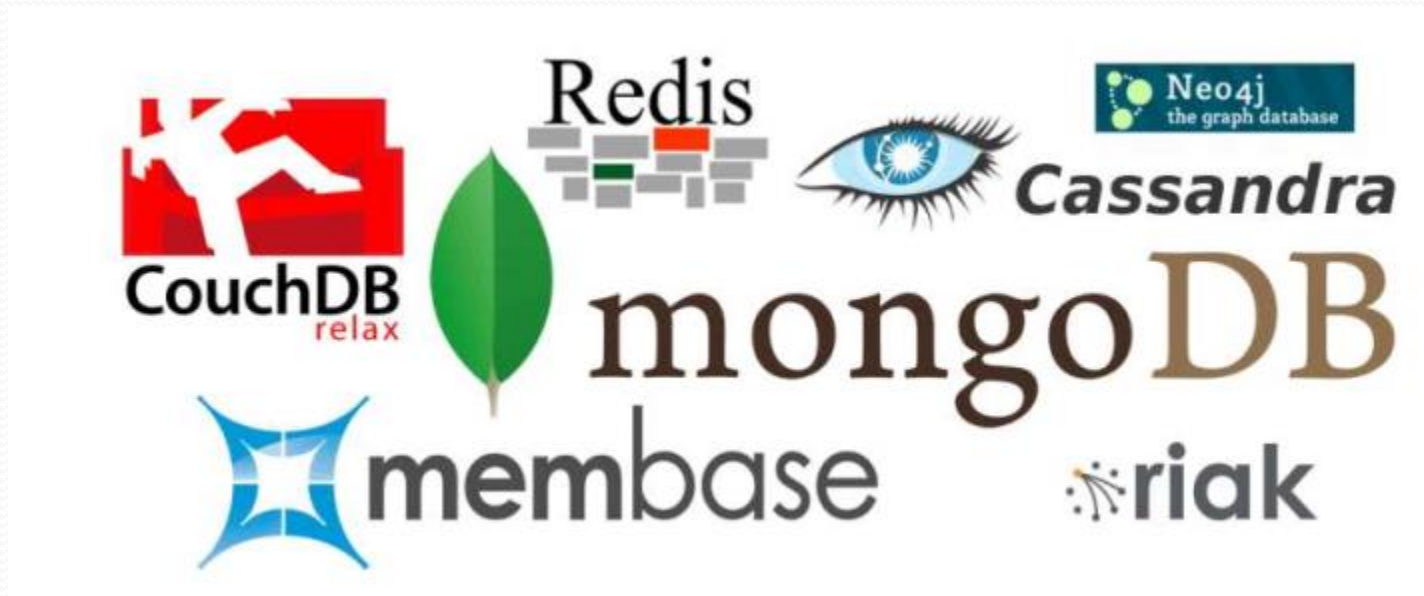
Por qué surgen las bases NoSQL

- Poca eficiencia en aplicaciones en las BD relacionales
- Aumento de operaciones de lectura y escritura
- Gran conjunto de transacciones
- Sentencias complejas
- Dificultades en la escalabilidad del sistema

Modelo no relacional

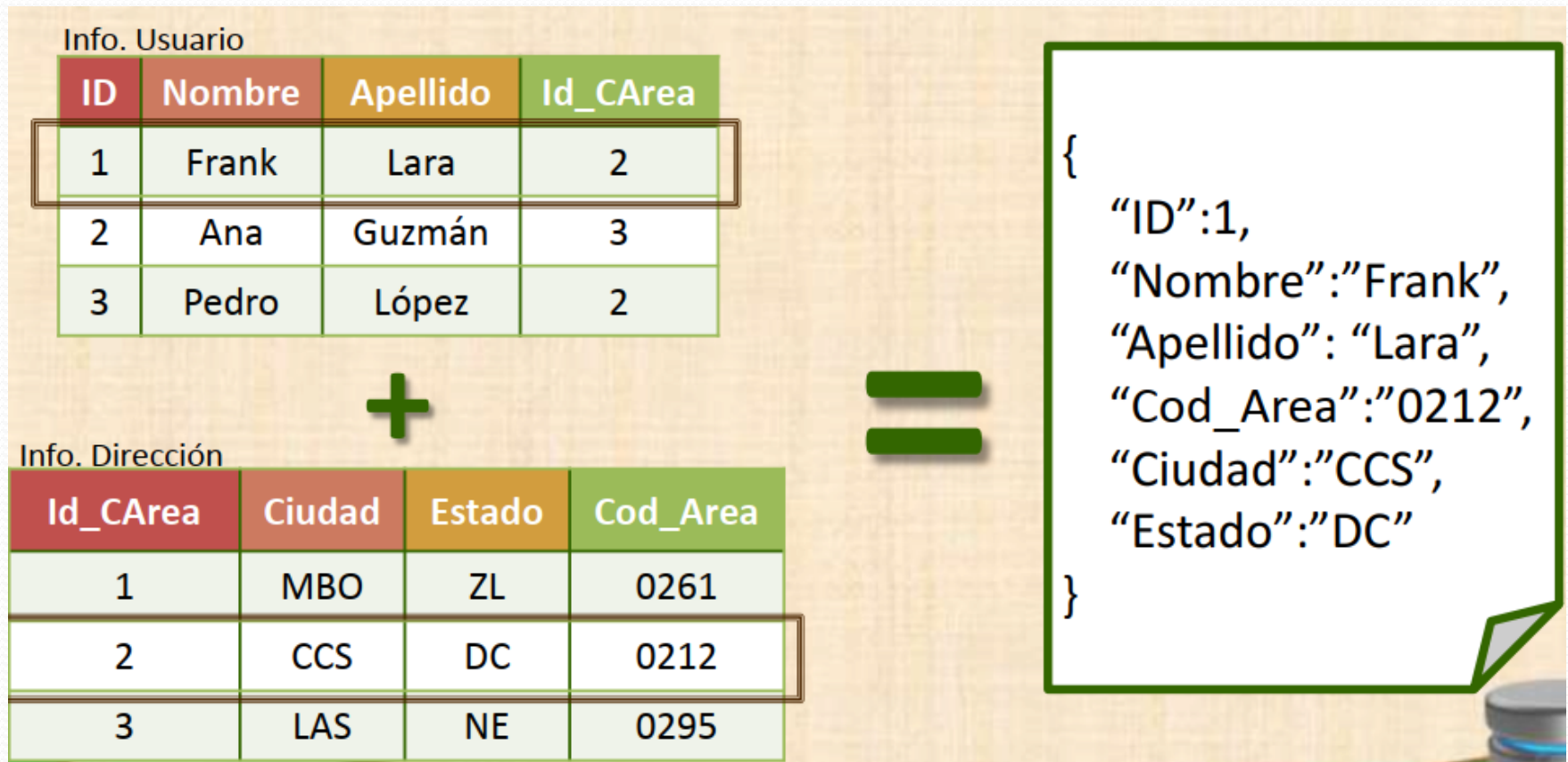
- Sistema de almacenamiento de información
- No cumple con el esquema entidad-relación
- No impone una estructura de datos
- Almacena los datos en diferentes formatos

Qué bases NoSQL conocen?



Propiedades

- Ausencia de esquema en los registros de datos:



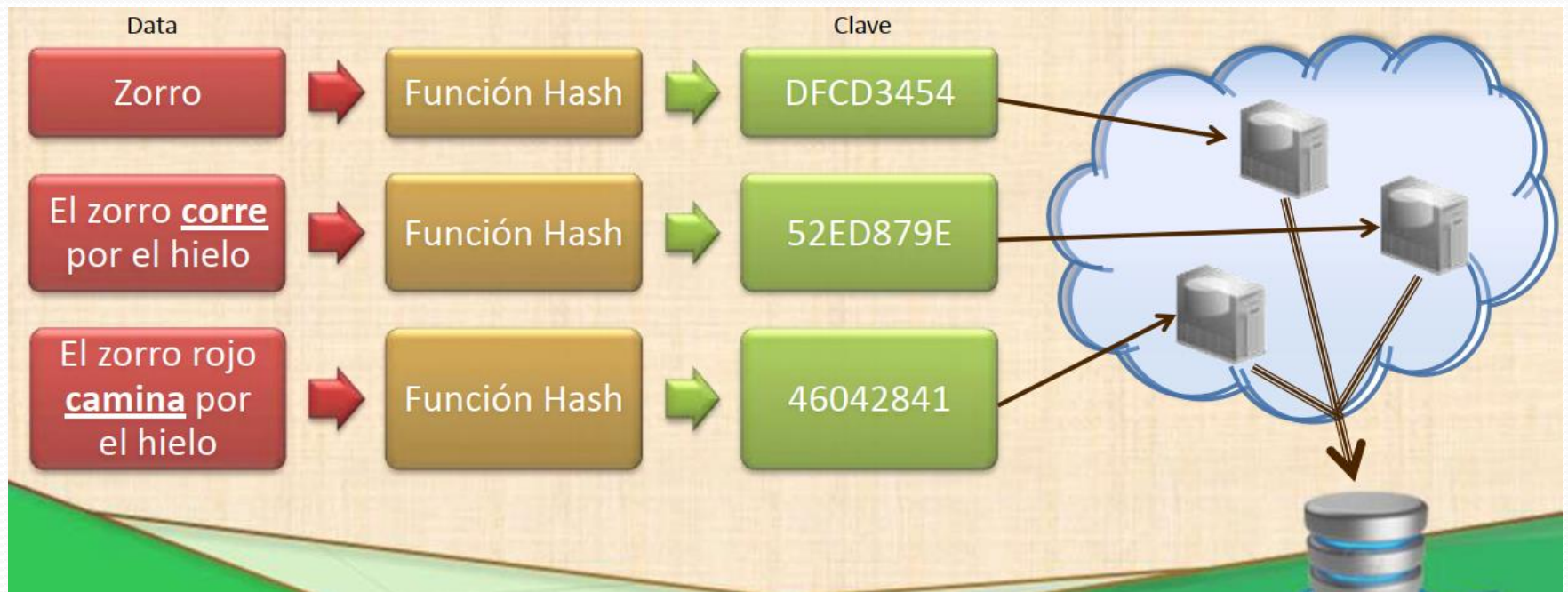
Propiedades

- Alta velocidad de respuesta a peticiones



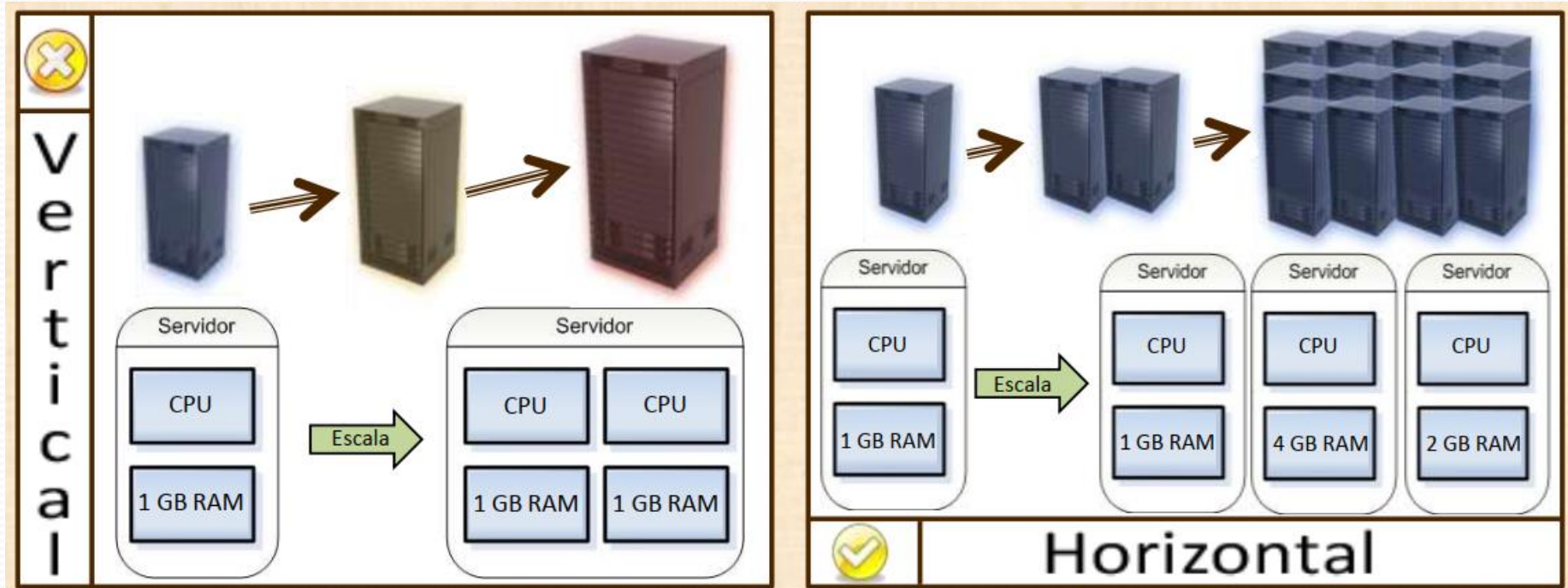
Propiedades

- Estructura distribuida



Propiedades

- Escalabilidad



Ventajas

- Estos sistemas responden a las necesidades de escalabilidad horizontal que tienen cada vez más empresas
- Pueden manejar enormes cantidades de datos
- No generan cuellos de botella
- Escalamiento sencillo
- Diferentes DBs NoSQL para diferentes proyectos
- Se ejecutan en clusters de máquinas baratas

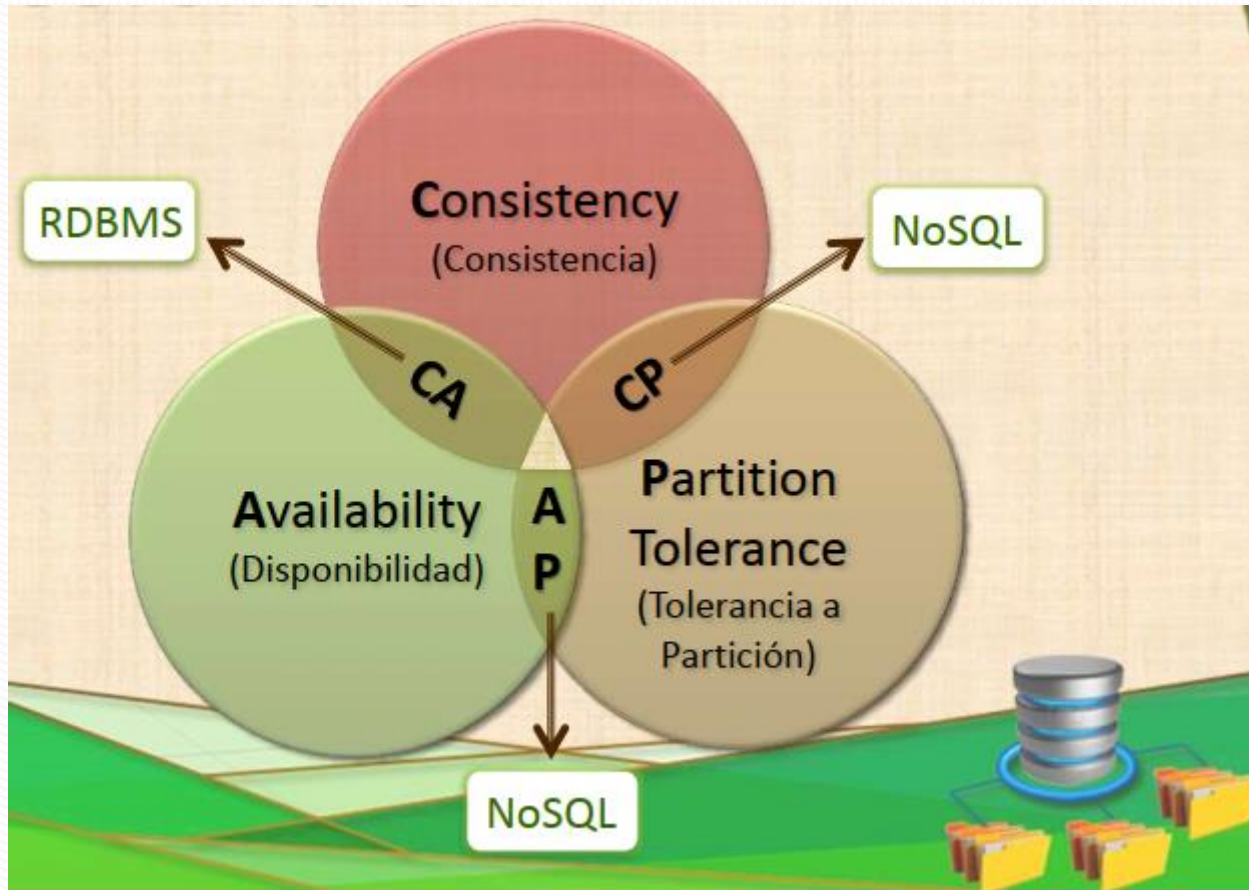
Desventajas

- No son suficientemente maduros para algunas empresas
- Falta de experiencia
- No trata con datos críticos que requieren ACID
- Problemas de compatibilidad

Teorema CAP

- Un compromiso fundamental que afecta a cualquier sistema distribuido obliga a los diseñadores de bases de datos a elegir solo dos de estas tres propiedades:
 - consistencia de datos
 - disponibilidad del sistema
 - o tolerancia a las particiones de red
- Esta limitación intrínseca de los sistemas distribuidos se conoce como Teorema CAP o teorema de Brewer:
 - Es imposible garantizar simultáneamente lo siguientes propiedades en un sistema distribuido:

CAP gráficamente



Consistencia

- Si se cumple este atributo, se garantiza que una vez que se escriben los datos están disponibles y actualizados para cada usuario que utilice el sistema

Disponibilidad

- Esta propiedad se refiere a ofrecer el servicio ininterrumpidamente y sin degradación dentro de un cierto porcentaje de tiempo

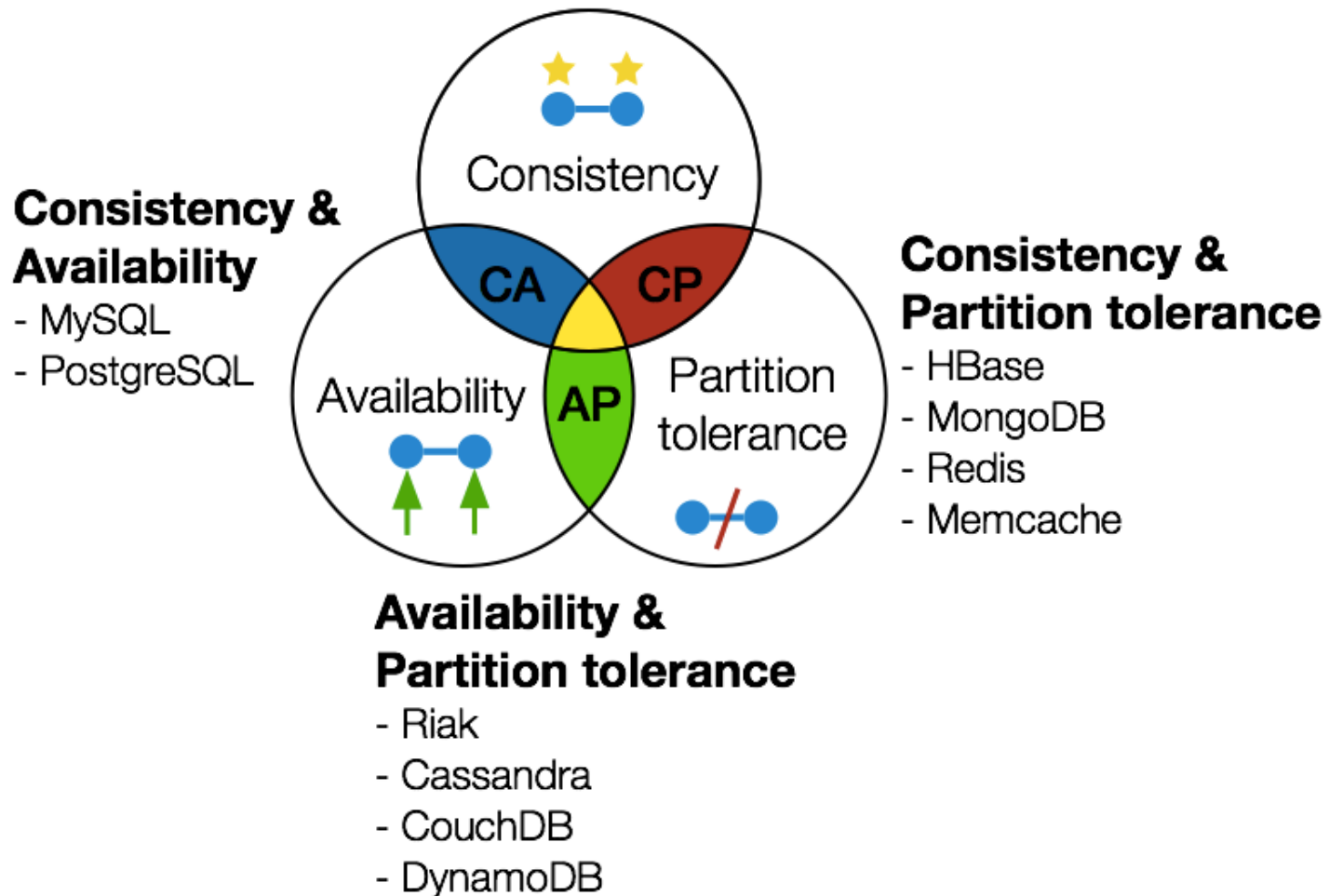
Tolerancia a la Partición

- Si el sistema cumple con esta propiedad entonces una operación puede ser completada incluso cuando alguna parte de la red falla

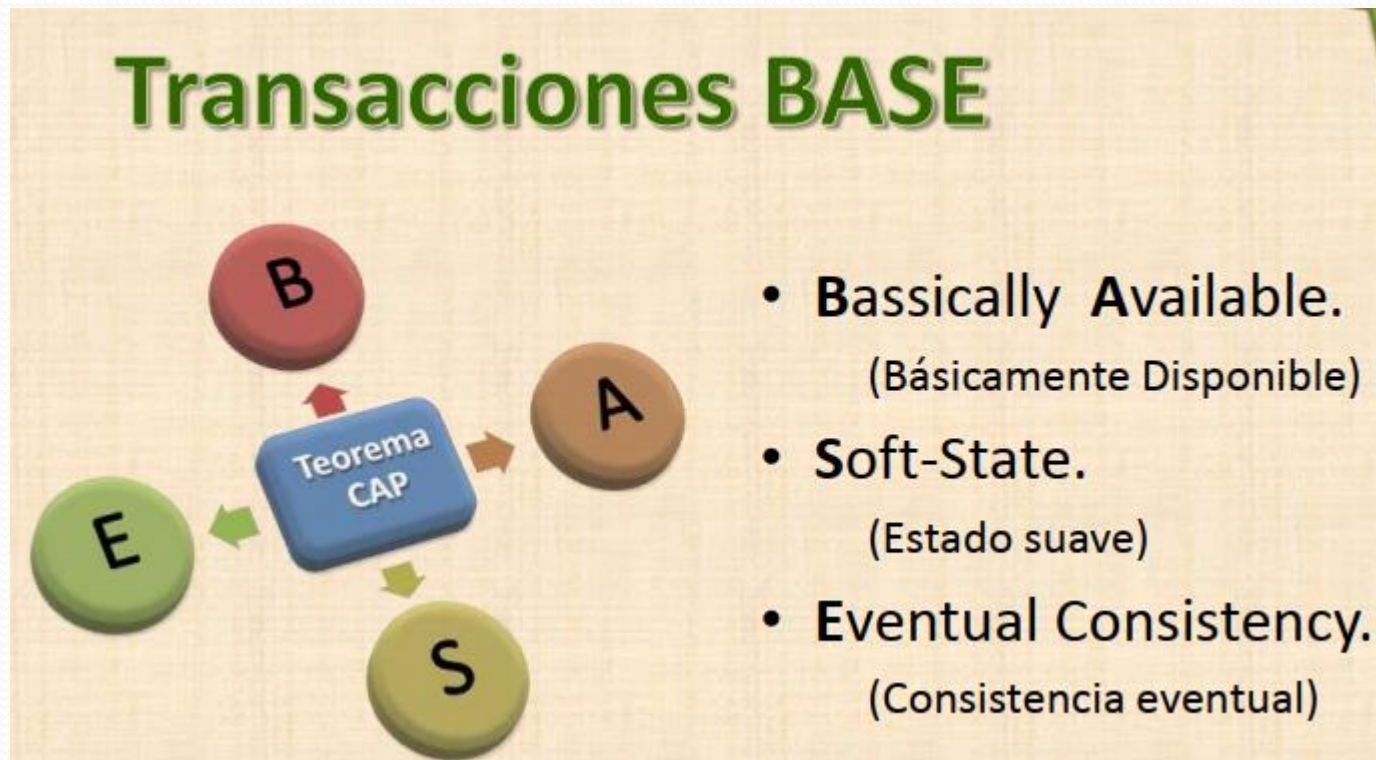
El teorema sólo garantiza

- **CP (Consistency & Partition)**: El sistema aplicará los cambios de forma **consistente** y aunque se pierda la comunicación entre nodos ocasionando el **particionado**, no se asegura que haya disponibilidad
- **AP (Availability & Partition)**: El sistema siempre estará **disponible** a las peticiones aunque se pierda la comunicación entre los nodos ocasionando el **particionado**, y en consecuencia por la pérdida de comunicación existirá inconsistencia porque no todos los nodos serán iguales
- **CA (Consistency & Availability)**: El sistema siempre estará **disponible** respondiendo las peticiones y los datos procesados serán **consistentes**. En este caso no se puede permitir el particionado

Clasificación según CAP



Transacciones BASE



Propiedad BASE

- Disponibilidad todo el tiempo, un estado flexible y consistencia eventual, por lo que en preciso momento puede que no sea consistente pero a lo largo del tiempo será **eventualmente consistente**
- En conclusión los sistemas ACID fuerzan la consistencia de la base de datos en todo momento sacrificando parte de la disponibilidad. Sin embargo **BASE gana disponibilidad** perdiendo parte de la consistencia, otorgando un estado de los datos **más flexible**

Taxonomía: cuatro categorías principales

Columna



Documento



Clave-valor



Grafo



BD orientadas a grafos

- La información es representada en nodos
- Ya está normalizada
- No es necesario definir cantidad de atributos
- Registros de longitud variable
- Algunas bases: Neo4j, HyperGraphDB

Ejemplo



BD Familia de Columnas

- Guarda los valores en columnas
- Los datos son almacenados como secciones de las columnas de datos

Ejemplo:

Orientado a Fila
(Modelo RDBMS)

Id	Nombre	Edad	Intereses
1	Ricky		Fútbol, Cine, Béisbol
2	Pedro	20	
3	Juan	25	Música

Multivalor

Null

Orientado a
Columnas

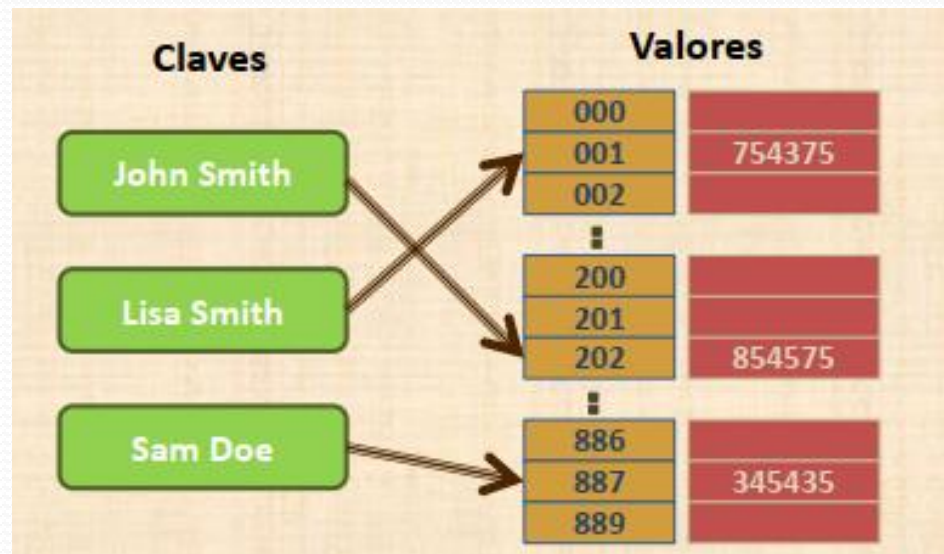
Id	Nombre
1	Ricky
2	Pedro
3	Juan

Id	Edad
2	20
3	25

Id	Intereses
1	Fútbol
1	Cine
1	Béisbol
3	Música

BD Clave-Valor

- Conjunto de duplas (Clave, Valor)
- Existen contenedores
- Permite variar la estructura de la información
- Validación de los datos en la aplicación Cliente

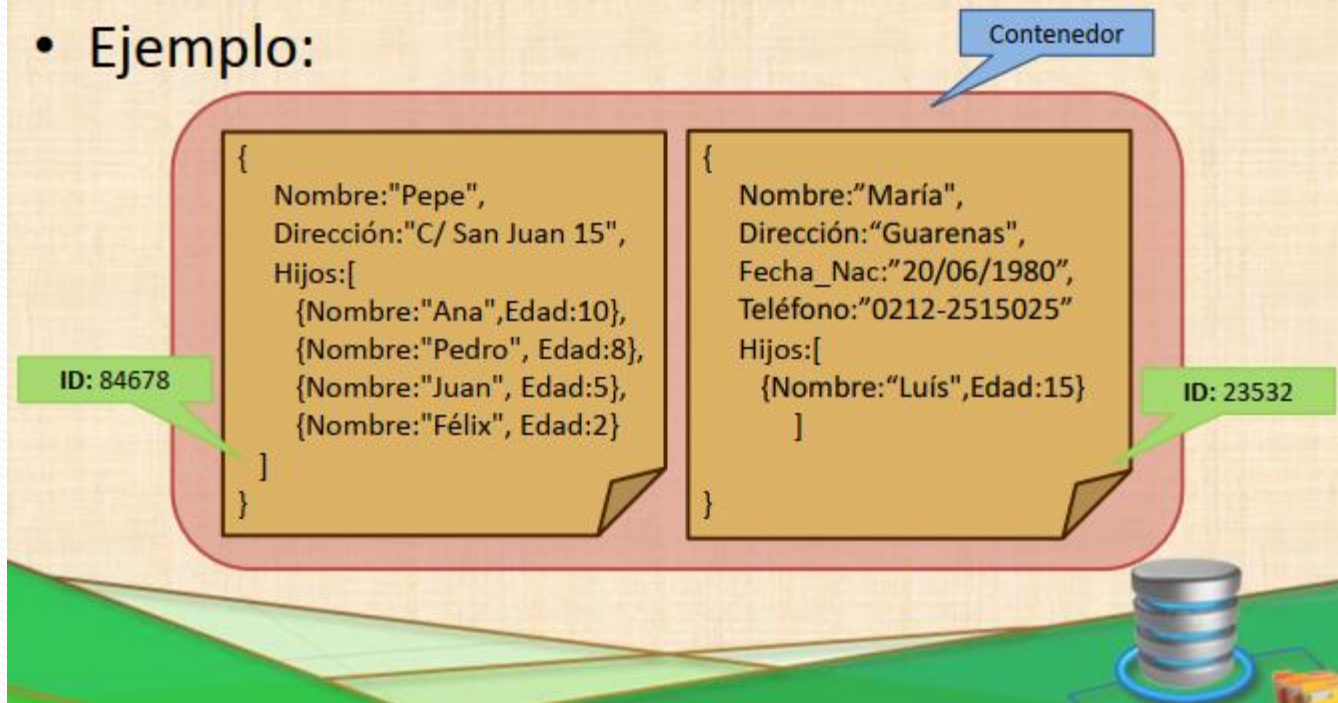


BD Orientada a Documentos

- Almacena los datos en documentos
- Son duplas Clave-(Valor=> documento)
- No existe un esquema estricto
- Los documentos dentro de una colección pueden tener campos diferentes

BD Orientada a Documentos

- Ejemplo:



¿Cuándo usar SQL o NoSQL?



MongoDB



- Escalable, alto rendimiento y disponibilidad
- Puede trabajar en modo maestro-esclavo
- Basada en esquemas BSON (*Binary JSON*)
- Posee un rico y sencillo sistema de consulta
- Soporte de índices
- Replicación y soporte a prueba de fallos

Propiedades

- Escalabilidad horizontal (Auto-Sharding)
- Replicación para alta disponibilidad



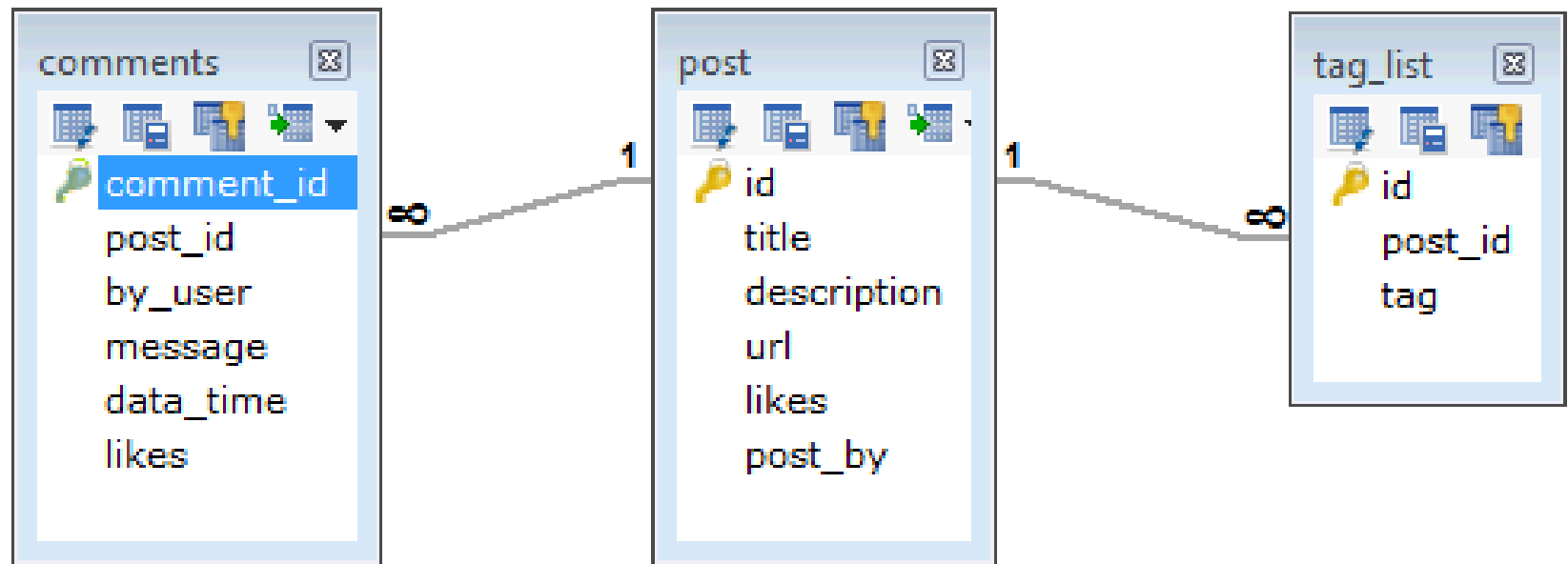
Arquitectura de MongoDB

- Los principales componentes de MongoDB son los siguientes:
 - Mongod (Núcleo de la base de datos)
 - Mongos (Controlador de particionamiento)
 - GridFS (Función de almacenamiento)

Comparación entre SQL y MongoDB

SQL	MongoDB
<code>CREATE TABLE users (id MEDIUMINT NOT NULL AUTO_INCREMENT, user_id Varchar(30), age Number, status char(1), PRIMARY KEY (id));</code>	<code>db.users.insert({user_id: "abc123", age: 55, status: "A"})db.createCollection("users")</code>
<code>DROP TABLE users</code>	<code>db.users.drop()</code>
<code>INSERT INTO users(user_id,age, status) VALUES ("bcd001", 45, "A")</code>	<code>db.users.insert({ user_id: "bcd001", age: 45, status: "A" })</code>
<code>SELECT * FROM users</code>	<code>db.users.find()</code>
<code>SELECT COUNT(*) FROM users</code>	<code>db.users.count()odb.users.find().count()</code>
<code>SELECT * FROM users WHERE status = "A" ORDER BY user_id DESC</code>	<code>db.users.find({ status: "A" }). sort({ user_id: -1 })</code>
<code>UPDATE users SET status = "C" WHERE age > 25</code>	<code>db.users.update({ age: { \$gt: 25 } }, { \$set: { status: "C" } }, { multi: true })</code>
<code>DELETE FROM users WHERE status = "D"</code>	<code>db.users.remove({ status: "D" })</code>

Migrando de SQL a NoSQL



En MongoDB

```
{
  _id: POST_ID
  title: TITLE_OF_POST,
  description: POST_DESCRIPTION,
  by: POST_BY,
  url: URL_OF_POST,
  tags: [TAG1, TAG2, TAG3],
  likes: TOTAL_LIKES,
  comments: [
    {
      user: 'COMMENT_BY',
      message: TEXT,
      dateCreated: DATE_TIME,
      like: LIKES
    },
    {
      user: 'COMMENT_BY',
      message: TEXT,
      dateCreated: DATE_TIME,
      like: LIKES
    }
  ]
}
```

Creación de Base de Datos

- `use DATABASE_NAME`
- Para chequear la base seleccionada:
 - `> db`
- Mostrar bases de datos corriendo:
 - `> show dbs`

Borrar Base de datos

- `db.dropDatabase()`

Colecciones

- `db.createCollection("myCollection")`
`{ "ok": 1 }`
- `db.myCollection.insert({"name": "tutorial"})`
- Borrar Colección
 - `db.myCollection.drop()`

Inserción de Documentos

```
>db.mycol.insert({  
  _id: ObjectId(7df78ad8902c),  
  title: 'MongoDB Overview',  
  description: 'MongoDB is no sql database',  
  by: 'tutorials point',  
  url: 'http://www.tutorialspoint.com',  
  tags: ['mongodb', 'database', 'NoSQL'],  
  likes: 100  
})
```

Inserción de Múltiples Documentos

```
>db.post.insert([
  {
    title: 'MongoDB Overview',
    description: 'MongoDB is no sql database',
    by: 'tutorials point',
    url: 'http://www.tutorialspoint.com',
    tags: ['mongodb', 'database', 'NoSQL'],
    likes: 100
  },

  {
    title: 'NoSQL Database',
    description: "NoSQL database doesn't have tables",
    by: 'tutorials point',
    url: 'http://www.tutorialspoint.com',
    tags: ['mongodb', 'database', 'NoSQL'],
    likes: 20,
    comments: [
      {
        user: 'user1',
        message: 'My first comment',
        dateCreated: new Date(2013,11,10,2,35),
        like: 0
      }
    ]
  }
])
```

Consultas en MongoDB

```
>db.mycol.find().pretty()  
{  
  "_id": ObjectId(7df78ad8902c),  
  "title": "MongoDB Overview",  
  "description": "MongoDB is no sql database",  
  "by": "tutorials point",  
  "url": "http://www.tutorialspoint.com",  
  "tags": ["mongodb", "database", "NoSQL"],  
  "likes": "100"  
}  
>
```

Consultas con “Where”

Operation	Syntax	Example	RDBMS Equivalent
Equality	{<key>: <value>}	db.mycol.find({"by":"tutorials point"}).pretty()	where by = 'tutorials point'
Less Than	{<key>: {\$lt: <value>}}	db.mycol.find({"likes": {\$lt:50}}).pretty()	where likes < 50
Less Than Equals	{<key>: {\$lte: <value>}}	db.mycol.find({"likes": {\$lte:50}}).pretty()	where likes <= 50
Greater Than	{<key>: {\$gt: <value>}}	db.mycol.find({"likes": {\$gt:50}}).pretty()	where likes > 50
Greater Than Equals	{<key>: {\$gte: <value>}}	db.mycol.find({"likes": {\$gte:50}}).pretty()	where likes >= 50
Not Equals	{<key>: {\$ne: <value>}}	db.mycol.find({"likes": {\$ne:50}}).pretty()	where likes != 50

Uso de AND

```
>db.mycol.find({$and:[{"by":"tutorials point"}, {"title": "MongoDB Overview"}]})
  "_id": ObjectId(7df78ad8902c),
  "title": "MongoDB Overview",
  "description": "MongoDB is no sql database",
  "by": "tutorials point",
  "url": "http://www.tutorialspoint.com",
  "tags": ["mongodb", "database", "NoSQL"],
  "likes": "100"
}
```

EN SQL: **where by = 'tutorials point' AND title = 'MongoDB Overview'**

Uso de OR

```
>db.mycol.find({$or:[{"by":"tutorials point"}, {"title": "MongoDB Overview"}]})
{
  "_id": ObjectId(7df78ad8902c),
  "title": "MongoDB Overview",
  "description": "MongoDB is no sql database",
  "by": "tutorials point",
  "url": "http://www.tutorialspoint.com",
  "tags": ["mongodb", "database", "NoSQL"],
  "likes": "100"
}
```

EN SQL: **where by = 'tutorials point' OR title = 'MongoDB Overview'**

AND y OR juntos

```
>db.mycol.find({"likes": {$gt:10}, $or: [{"by": "tutorials point"},  
  {"title": "MongoDB Overview"}]}).pretty()  
{  
  "_id": ObjectId(7df78ad8902c),  
  "title": "MongoDB Overview",  
  "description": "MongoDB is no sql database",  
  "by": "tutorials point",  
  "url": "http://www.tutorialspoint.com",  
  "tags": ["mongodb", "database", "NoSQL"],  
  "likes": "100"  
}  
>
```

En SQL: **where likes>10 AND (by = 'tutorials point' OR title = 'MongoDB Overview')**

Ordenamiento

- `>db.myCollection.find().sort({name: 1, surname: -1})`

Función de Agregación

```
{
  _id: ObjectId(7df78ad8902c)
  title: 'MongoDB Overview',
  description: 'MongoDB is no sql database',
  by_user: 'tutorials point',
  url: 'http://www.tutorialspoint.com',
  tags: ['mongodb', 'database', 'NoSQL'],
  likes: 100
},
{
  _id: ObjectId(7df78ad8902d)
  title: 'NoSQL Overview',
  description: 'No sql database is very fast',
  by_user: 'tutorials point',
  url: 'http://www.tutorialspoint.com',
  tags: ['mongodb', 'database', 'NoSQL'],
  likes: 10
},
{
  _id: ObjectId(7df78ad8902e)
  title: 'Neo4j Overview',
  description: 'Neo4j is no sql database',
  by_user: 'Neo4j',
  url: 'http://www.neo4j.com',
  tags: ['neo4j', 'database', 'NoSQL'],
  likes: 750
},
```

Función de Agregación

```
> db.mycol.aggregate([{$group : {_id : "$by_user", num_tutorial : {$sum : 1}}}])
{
  "result" : [
    {
      "_id" : "tutorials point",
      "num_tutorial" : 2
    },
    {
      "_id" : "Neo4j",
      "num_tutorial" : 1
    }
  ],
  "ok" : 1
}
```

En SQL: **select by_user, count(*) from mycol group by by_user.**

Ensamble de Colecciones

post collection

```
{
  "title" : "my first post",
  "author" : "Jim",
  "likes" : 5
},
{
  "title" : "my second post",
  "author" : "Jim",
  "likes" : 2
},
{
  "title" : "hello world",
  "author" : "Joe",
  "likes" : 3
}
```

comment collection

```
{
  "postTitle" : "my first post",
  "comment" : "great read",
  "likes" : 3
},
{
  "postTitle" : "my second post",
  "comment" : "good info",
  "likes" : 0
},
{
  "postTitle" : "my second post",
  "comment" : "i liked this post",
  "likes" : 12
},
```

Ensamble de Colecciones (cont.)

```
db.posts.aggregate([
  { $lookup:
    {
      from: "comments",
      localField: "title",
      foreignField: "postTitle",
      as: "comments"
    }
  }
])
```

Vistas en MongoDB

- Creación de vistas

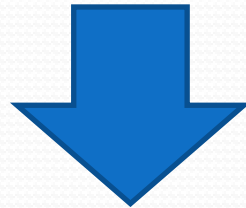
```
db.createCollection(  
  "<viewName>",  
  {  
    "viewOn" : "<source>",  
    "pipeline" : [<pipeline>],  
    "collation" : { <collation> }  
  }  
)
```

```
db.createView(  
  "<viewName>",  
  "<source>",  
  [<pipeline>],  
  {  
    "collation" : { <collation> }  
  }  
)
```

Ejemplo de Vistas

Collection: Survey

```
[ { _id: 1, empNumber: "abc123", feedback: { management: 3, environment: 3 }, department: "A"
  { _id: 2, empNumber: "xyz987", feedback: { management: 2, environment: 3 }, department: "B"
  { _id: 3, empNumber: "ijk555", feedback: { management: 3, environment: 4 }, department: "A"
```



View: managementFeedback

```
db.createView(
  "managementFeedback",
  "survey",
  [ { $project: { "management": "$feedback.management", department: 1 } } ]
)
```

Uso de la Vista

```
db.managementFeedback.find()
```

The operation returns the following documents:

```
{ "_id" : 1, "department" : "A", "management" : 3 }  
{ "_id" : 2, "department" : "B", "management" : 2 }  
{ "_id" : 3, "department" : "A", "management" : 3 }
```